WEEK 6 PROJECT

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Database Design, Review Operations & Rating Analysis

Subject	Database Management System (MySQL)
Project	Week 6 – Product Review and Rating Management System
Database	ecommerce
Relationships	Customer → Review; Product → Review

This report includes the Review table design, entity relationships, sample data, review operations, and product/customer rating analysis.

1. Objective

To extend the existing ecommerce database with product reviews and customer ratings. Each review references an existing customer and product, and ratings are limited to one through five stars.

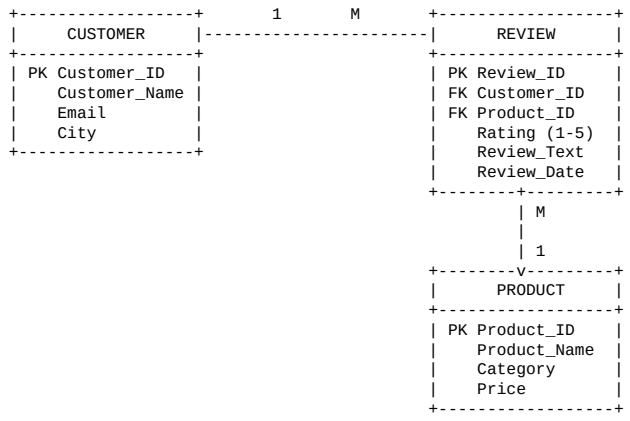
2. Review Table Design

Column	Data Type	Constraint / Purpose
Review_ID	INT	Primary key, auto-increment
Customer_ID	INT	NOT NULL, FK to Customer.Customer_ID
Product_ID	INT	NOT NULL, FK to Product.Product_ID
Rating	INT	NOT NULL, CHECK Rating BETWEEN 1 AND 5
Review_Text	VARCHAR(1000)	NOT NULL, stores customer feedback
Review_Date	DATETIME	NOT NULL, defaults to current timestamp

Constraint summary

• Primary Key: Review_ID uniquely identifies each review.
• Foreign Keys: Customer_ID and Product_ID ensure every review links to an existing customer and product.
• NOT NULL: Required IDs, rating, review text, and date cannot be omitted.
• CHECK: Rating must be an integer from 1 to 5.
• Referential actions: ON DELETE CASCADE removes a customer's/product's associated reviews if that parent row is deleted. Change to RESTRICT if your project rules require preserving review history.

3. Updated ER Diagram




```
Customer 1 ---- M Review
Product  1 ---- M Review
```

4. Sample Customer, Product & Review Data

The sample review records below reference Customer_ID and Product_ID values from the Week 4 ecommerce database. Run the Week 4 SQL script first.

Review_ID	Customer_ID	Product_ID	Rating	Review date	Review text
1	1	101	5	2026-10-02	Excellent laptop performance.
2	2	102	4	2026-10-04	Comfortable mouse; good value.
3	3	101	4	2026-10-13	Works well for daily tasks.
4	4	103	3	2026-10-16	Keys feel a little stiff.
5	1	104	5	2026-10-17	Clear sound and comfortable fit.
6	2	105	2	2026-10-18	Transfer speed is slower than expected.
7	3	102	5	2026-10-20	Easy to use and responsive.

Review_ID	Customer_ID	Product_ID	Rating	Review date	Review text
8	4	101	5	2026-10-22	Very good product; satisfied.

5. Product Review Operations

The complete executable MySQL script is provided separately as Week6_Product_Review_Rating_Management.sql. Key queries are shown below.

Create the Review table

```
CREATE TABLE Review (
  Review_ID INT PRIMARY KEY AUTO_INCREMENT,
  Customer_ID INT NOT NULL,
  Product_ID INT NOT NULL,
  Rating INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
  Review_Text VARCHAR(1000) NOT NULL,
  Review_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
    ON UPDATE CASCADE ON DELETE CASCADE
);
```

Insert, display, update and delete reviews

```
INSERT INTO Review (Customer_ID, Product_ID, Rating, Review_Text)
VALUES (2, 104, 4, 'Good sound and comfortable to wear.');

SELECT c.Customer_Name, p.Product_Name, r.Rating,
       r.Review_Text, r.Review_Date
FROM Review r
JOIN Customer c ON c.Customer_ID = r.Customer_ID
JOIN Product p ON p.Product_ID = r.Product_ID
ORDER BY r.Review_Date DESC;

UPDATE Review
SET Review_Text = 'Updated: excellent product and reliable performance.',
    Rating = 5
WHERE Review_ID = 3;

-- Review first; uncomment only when deletion is appropriate:
-- DELETE FROM Review WHERE Review_ID = 6;
```

Display reviews for one product and ratings above 4

```
SELECT p.Product_Name, c.Customer_Name, r.Rating,
       r.Review_Text, r.Review_Date
FROM Review r
JOIN Product p ON p.Product_ID = r.Product_ID
JOIN Customer c ON c.Customer_ID = r.Customer_ID
WHERE r.Product_ID = 101
ORDER BY r.Review_Date DESC;

SELECT * FROM Review WHERE Rating > 4
ORDER BY Review_Date DESC;
```

6. Product Rating Analysis & Reports

```
-- Average rating and number of reviews per product
SELECT p.Product_Name, COUNT(r.Review_ID) AS Number_Of_Reviews,
       ROUND(AVG(r.Rating),2) AS Average_Rating
FROM Product p LEFT JOIN Review r ON r.Product_ID=p.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Average_Rating DESC;

-- Products with average rating above 4
SELECT p.Product_Name, ROUND(AVG(r.Rating),2) AS Average_Rating
FROM Product p JOIN Review r ON r.Product_ID=p.Product_ID
GROUP BY p.Product_ID, p.Product_Name
HAVING AVG(r.Rating)>4;

-- Products requiring improvement (average rating below 3.5)
SELECT p.Product_Name, ROUND(AVG(r.Rating),2) AS Average_Rating
FROM Product p JOIN Review r ON r.Product_ID=p.Product_ID
GROUP BY p.Product_ID, p.Product_Name
HAVING AVG(r.Rating)<3.5;

-- Count of five-star, four-star and low-rated (1-3) reviews
SELECT SUM(CASE WHEN Rating=5 THEN 1 ELSE 0 END) AS Five_Star,
       SUM(CASE WHEN Rating=4 THEN 1 ELSE 0 END) AS Four_Star,
       SUM(CASE WHEN Rating<=3 THEN 1 ELSE 0 END) AS Low_Rated
FROM Review;

-- Customer feedback analysis
SELECT c.Customer_Name, COUNT(r.Review_ID) AS Reviews_Written,
       ROUND(AVG(r.Rating),2) AS Average_Rating_Given,
       SUM(CASE WHEN r.Rating>=4 THEN 1 ELSE 0 END) AS Positive_Reviews,
       SUM(CASE WHEN r.Rating<=2 THEN 1 ELSE 0 END) AS Negative_Reviews
FROM Customer c LEFT JOIN Review r ON r.Customer_ID=c.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Reviews_Written DESC;
```

7. Illustrative Rating Analysis Outputs

These results are calculated from the eight sample reviews before running the optional insert/update/delete examples. They are illustrative outputs, not screenshots from a live MySQL Workbench session.

Product	Review count	Average rating
Laptop (101)	3	4.67
Wireless Mouse (102)	2	4.50
Headphones (104)	1	5.00
Keyboard (103)	1	3.00
USB Drive 64GB (105)	1	2.00

Rating distribution

Rating group	Count
5-star reviews	3
4-star reviews	2
Low-rated reviews (1–3 stars)	3

Products requiring improvement (average rating below 3.5): Keyboard (3.00) and USB Drive 64GB (2.00). Highest-rated product by average rating: Headphones (5.00), based on one review.

8. How to Run and Submit

1. Run the Week 4 ecommerce script first so the Customer and Product tables exist.
2. Open Week6_Product_Review_Rating_Management.sql in MySQL Workbench and execute the CREATE TABLE and sample INSERT sections.
3. Run each operation and report query. Do not repeat sample INSERTs after they have succeeded, because Review_ID values are primary keys.
4. Capture actual output screenshots and add your name/college details if required.
5. Upload the PDF and SQL script to GitHub, then submit the repository URL using your instructor's Google Form.

Important: The script uses MySQL 8.0+ for CHECK constraint support. The report uses sample data for illustration; run the queries in your own database to capture actual screenshots.